

Predicting Apple Inc. stock using Multilayer perceptron

Thomas Smeman S4950836
Gideon Jadael Wiersma S4979796
Petar I. Penchev S4683099

October 2024

Abstract

This report explores the optimal hyperparameters for constructing a Multilayer Perceptron (MLP) to aid in predicting the next closing price of Apple Inc. stock. A pipeline is used where the residuals between the trend and actual closing prices serve as input to the MLP. The MLP model is tuned through a global search over all hyperparameter permutations. The tuned hyperparameters are: the number of neurons per layer, the batch size and learning rate. Statistical analysis provides insights into the influence of parameter choices on prediction accuracy.

1 Introduction

1.1 Research Question

What is the best set of hyperparameters to use for construction of an MLP, that is an intrinsic part of a pipeline, designed to predict the next closing price of a financial asset? In our case the asset in question is Apple Inc..

1.2 Approach

In this report, an attempt is made to predict a sequence of closing prices of the stock market. To accomplish this, a simple pipeline is employed with multilayer perceptron (MLP) as its main feature. First, the general trend of the market is determined with a simple moving average (SMA). Then the residuals between the SMA and the closing prices are computed. This serves as the training data for our feed forward neural network. The trend is then extrapolated using simple linear regression, and the predictions of the MLP are overlayed on it to give us the closing prices.

An MLP works by manipulating the input vector (a number of residuals in our case) with a series of weight matrices and sigmoid functions in an alternating manner. After each matrix multiplication and sigmoid appliance, a new layer

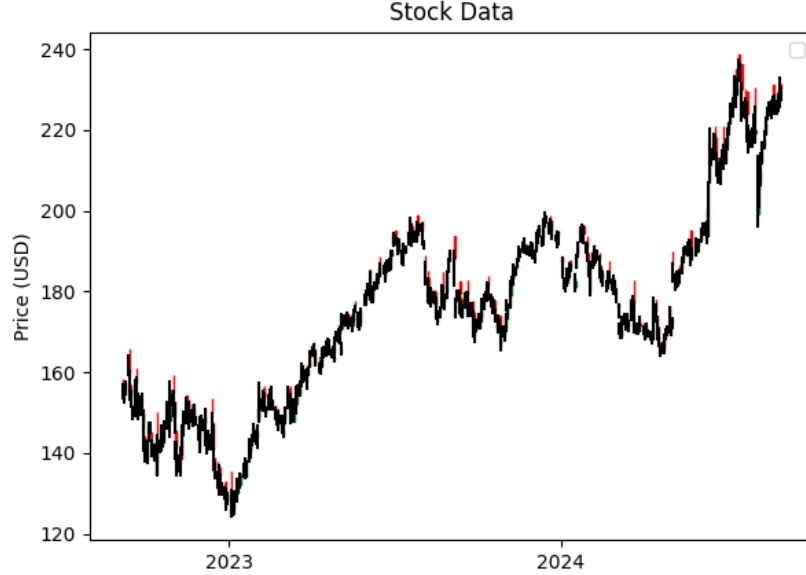


Figure 1: The data used for this project. This graph shows the candlestick diagram of price of shares (in USD) of Apple Inc, from May 20th, 2023 till October 1st, 2024. Note that only the closing prices are used for analysis.

is formed with the last layer being the output layer. In our case, a choice has been made for a single neuron output, estimating the new residual. The ideal MLP is found by exploring the hyperparameter space via a global search over all possible permutations of hyperparameter combinations. Then a statistical analysis is performed to give us some insight into how the topology of the hyperparameter space influences the loss function. The chosen model is the one that minimizes the mean average error (MAE) after cross-validation.

2 Data

The dataset used for this project comes from Yahoo Finance’s API (Yahoo, 2024) via the python package (Aroussi, 2024). For the analysis, the last 500 closing prices of the shares of Apple Inc. have been used at a candle interval of 1 day. The data is split up into 50 sets of 10 points. For each set, the first 9 prices are used as input features and the last price as the output feature. See Figure 1 for a visualization of the data.

3 Methods and experiments

Residuals

The main problem in trying to make any financial predictions is that wildly unpredictable factors can influence the market and thus the next stock price. The initial idea of predicting the closing prices was substituted with a slightly different approach, namely to try to predict the residuals. The residuals are the difference between the closing prices and the trend of the stock data. This trend is computed using a simple moving average through the closing prices:

$$\text{Residual}(\text{date}) = \text{Price}(\text{date}) - \text{SMA}(\text{date}), \quad (1)$$

where $\text{Price}(\text{date})$ is the closing price at a particular date and $\text{SMA}(\text{date})$ is the simple moving average, thus the value of the trend, of a particular date. By computing the residuals for each point, we gain a sequence of the residuals. By splitting the last entry from the rest, we obtain chunks of 9 data-points with one label, as the aim is to predict the next point. By using the sliding window technique, an indefinite amount of predictions can be made.

Sliding Window

The way this works is that the next residual is predicted based on the last 9 residuals. Then, the window is slid one to the right by removing the first value from the input and adding the predicted value to the end of the input. A new residual is then predicted based on this new input. This process is iterated till the desired amount of predictions is made.

Important note on the sliding window is that it suffers all the caveats that any technique involving bootstrapping has. This is especially exacerbated by the fact that our residuals are, at least in theory, correlated across time and bootstrapping assumes that observations are independent.

Data Split

After pre-processing the data by extracting the residual, the data is split up into *training* (80 %), *validation* (10 %), and *testing* (10 %) datasets. The function of these different datasets will be explained further down.

Finding Ideal Weights

To obtain the single price prediction, an MLP is trained on the residuals. We use tensorflow (Martín Abadi et al., 2015) and keras (Chollet et al., 2015) to achieve this. For the gradient descent we used the Adam optimizer (Kingma and Ba, 2019). This optimizer alters the standard gradient descent to solve its numerous numerical difficulties. The aim is thus to find the optimal set of weights θ_{opt} for a given model, given by

$$\theta_{\text{opt}} = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum L(\mathcal{N}_{\theta}(\mathbf{u}_i), y_i), \quad (2)$$

for $(\mathbf{u}_i, y_i) \in S$, where S is the set of all input/output of the past and the future, L is the loss function, the simple *mean squared error* in our case, $\mathcal{N}$ in the model

that transforms the input into the output, θ are the weights of the MLP, $\mathbf{u}_i$ is the sequence of residuals and y_i is the corresponding next residual in the data set.

Early Stopping

In real life the dataset S is not available, because the future is not perfectly predictable. To mitigate this problem, we say $S = T$ where T is the training data. Then it becomes possible, using gradient descent, to at least approach a local minimum. However, doing this for a limited dataset will always cause overfitting. To prevent this *early stopping* is used.

After each step in the gradient descent the sum of the loss function (the so-called *risk*) is again computed but now on a so-called *validation* set (V), which is completely distinct from T . If the performance of the model on the validation data sees no improvements for an arbitrary number of epochs, in our case we randomly picked 4 epochs, the training is stopped before the maximum number of 50 epochs is reached. We do this because if there is no improvement seen on validation data, but the model keeps improving in training, this is a clear indication that the model is memorizing the training data.

Batch Size

In each optimization step a gradient of the risk with respect to all the individual weights is calculated. However, to compute it, you need to get the gradient of the loss function of all the input vectors and their corresponding output vectors, which is computationally expensive. To solve this problem, the gradient is calculated as only the sum of some of the input/output data instead of updating the weight matrix after every step. The amount of data-points taken into account for every update of the weight matrix is called the batch size.

Hyper Parameters

To test how well the model is trained, one can test the model on the testing data, which is another dataset distinct of both the training and the validation data. Let the performance of the model on this data set be defined as

$$R_{\chi}^{emp} = \frac{1}{N} \sum_{i=1}^N L(\mathcal{N}_{\theta_{\chi}}(\mathbf{u}_i), y_i), \quad (3)$$

where $(\mathbf{u}_i, y_i) \in T_{test}$ and T_{test} is the testing data (with N elements). This time, the loss function is the *mean absolute error*, since that is commonly the most suitable for predicting continuous outputs. θ_{χ} denotes the set of weights found with a specific set of hyperparameters χ . See Figure 2 for the pipeline used to calculate equation 3.

To find which set of hyperparameters (χ_{opt}) creates the best performing model, the following minimization problem needs to be solved:

$$\chi_{opt} = \arg \min_{\chi \in X} R_{\chi}^{emp}. \quad (4)$$

The hyperparameter search space (X) for this research will include varying network architectures, where we use either one or two hidden layers with a varying number of neurons in each layer, varying learning rates, and varying batch sizes.

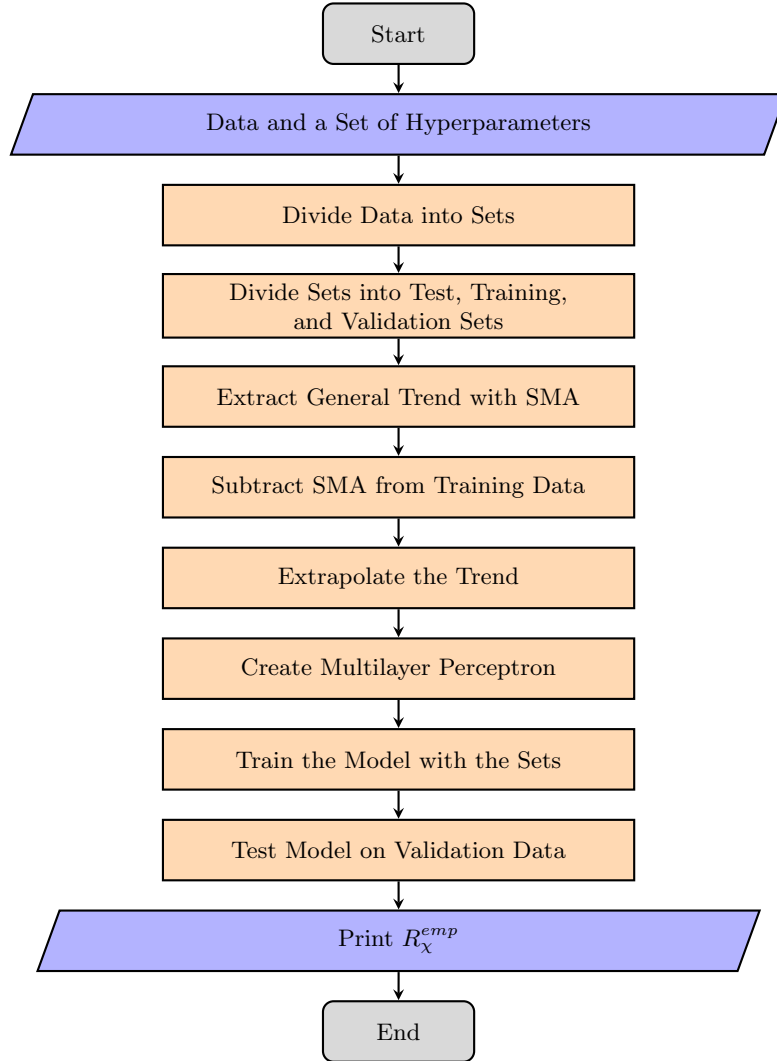


Figure 2: Getting the empirical risk for a set of hyperparameters.

4 Results

4.1 Getting Data for Analysing the Hyperparameter Space

In order to do this analysis a set of 5700 permutations of different parameter combinations have been used to train different models. A choice of a simple architecture have been made to reflect the randomness of the problem. To achieve this gigantic search, a 50 core multiprocessing job script was sent to the High Performance Computing cluster (Hábrók).

One or two hidden layers have been used, with either 4 to 16 neurons in the first layer or 0 to 16 neurons in the second layer (if it exists). The learning rates vary from 0.00050 to 0.01000 along with the batchsize in integer steps between 1 and 5. We kept the size of the input and the output constant at 9 and 1. We utilised the same transition function between hidden layers, namely ReLu with an exception for the output where we used linear.

Table 4 gives an overview of the parameters explored. A scatter-plot matrix was also made to aid in the visualization of the explored parameters and how they influence each other (see Figure 6).

4.2 Analysis

The correlation coefficients of the hyperparameters with respect to the MAE (also known as R_{χ}^{emp}) can be seen in Table 2. The associated p-values are also available in Table 3, and is evident (from their low value of 0 for all) that there is a strong correlation between the hyperparameters and the MAE, as expected. The following relationships can be seen:

- The number of neurons in layer one is slightly positively correlated with MAE (the "badness") of the model.
- The number of neurons in layer 2 is weakly negatively correlated with the empirical risk. R_{χ}^{emp} .
- The learning rate, is negatively correlated.
- A higher batch size seems to have low negative correlation.
- The number of layers is also negatively correlated.

Conclusion of the analysis:

- The more neurons in layer 1, the worse the model performs. That is we need less than the mean of the population which can be seen in Table 4 to be 10.
- The more neurons in layer 2, the better the model performs. Meaning less than $8.75 \approx 9$.
- The lower the learning rate, the better the model performs. Meaning less than 0.00525.

- Higher batch sizes could be better but the effect has diminishing returns
- Two layers seems slight better than one layer.

A nice visual overview can be seen in Figure 3. As expected none of the hyperparameters are correlated with each other, as they are independent variables, and the only correlation of course is to the MAE. A collection of the top 5 models, rated on performance, can be seen in the next chapter, alongside with the motivation of choosing one of them.

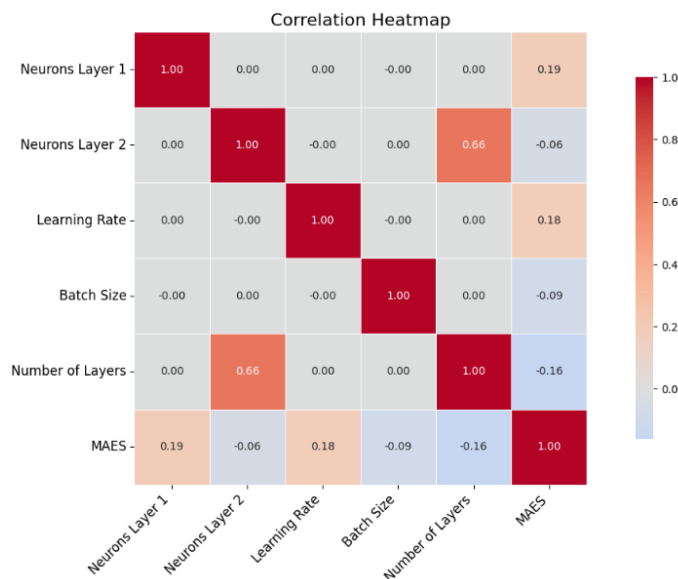


Figure 3: Heat map showing the correlation between individual hyperparameter pairs.

4.3 Top 5 best performing models

By looking at the the best 5 models (displayed in Table 1) we see that simple models are preferred over complex models. Either there is only one layer or the number of neurons per layer are similar in number. The apparent simplicity of those top 5 models is reflecting the randomness of the problem. Simple statistical techniques have been proven in the literature to be better than large neural networks (Chen, 2023), which explains why we get such a simple models after evaluation. It has also been shown that such neural networks are universal approximators (Hornik et al., 1989), which include those same statistical techniques, further facilitating the case for small neuron size and the inclusion of a few hidden layers.

Neurons Layer 1	Neurons Layer 2	Learning Rate	Batch Size	Number of Layers	R_{χ}^{emp}
8	0	0.0035	3	1	0.323205
16	16	0.0015	5	2	0.369153
6	0	0.0005	3	1	0.390235
4	4	0.0060	2	2	0.392130
16	14	0.0010	3	2	0.412513

Table 1: Top 5 Neural Networks (NNs) with Varying Architectures and Hyperparameters

4.4 Optimal Model Selection and Prediction Process

There is a clear winner from the analysis performed above and that is the model with a single hidden layer of 8 neurons, learning rate of 0.0035, and a batch size of 3.

This model is used to predict the next N residuals utilizing the sliding window method. In order to find the trend (i.e. the extrapolated SMA) a simple linear regression is being used on the last N points of the SMA (in our case $N = 5$ as we try to predict 5 prices ahead, the choice here is arbitrary).

The intermediate linear regression allows us to make the transition from residuals to closing prices, but it is important to note that this does not have any impact on the task of our MLP, namely to predict the residuals. If one wishes to judge the goodness of the model the figure that needs attention is the one on the bottom of Figure 5, where it can be seen how closely the model fits the data that hasn't been seen in training (the model is trained on the data before October 1st). Also an overview of the stock data along with overlaid predictions and extrapolated trend can be seen in Figure 4. Figure 4 and 5 show a visualization of the model.

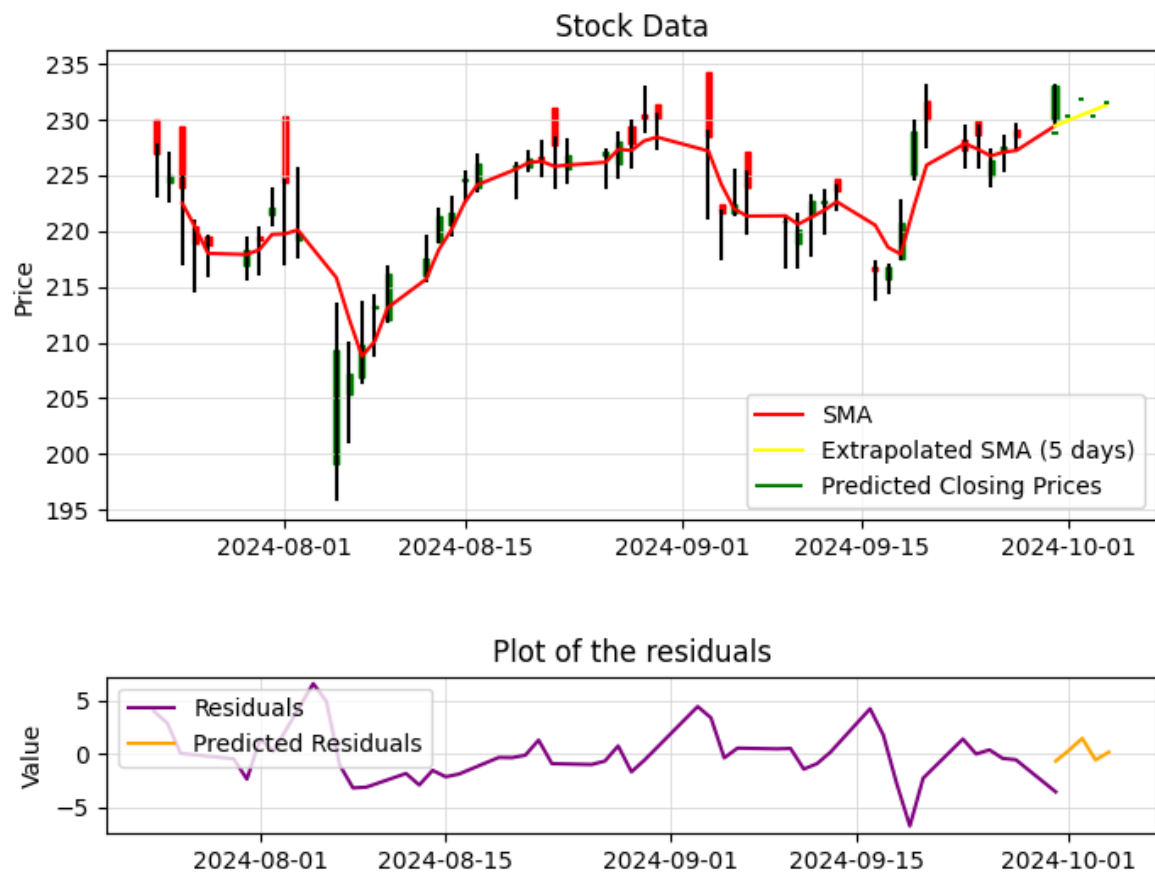


Figure 4: The stock market of Apple Inc, until October 1st and the predicted values after that date along with the extrapolated trend. Upper graph shows the entire stock data, while the lower graphs shows the residuals.

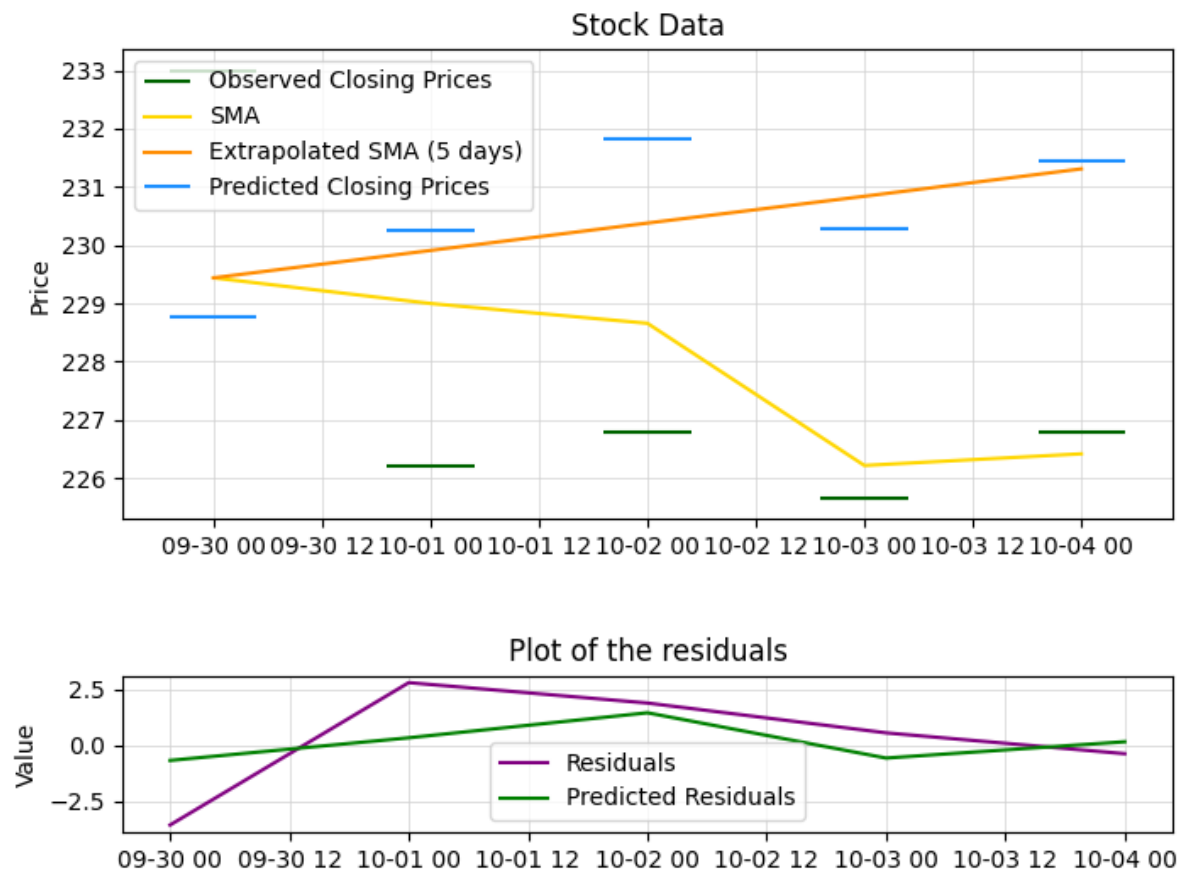


Figure 5: A comparison between the the predicted closing prices after October 1st and the actual prices after that date.

5 Discussion

There are a lot of free variables that could have been taken into account in training. As outlined above, the learning rate, network architecture, and the batch size are one of the most important ones, that’s why a choice to explore those was made. The number of input and output neurons were kept constant. The reasoning behind this was to constrain the volume of the search space. Two more parameters would have increased it exponentially, something that would have made exploration of said space even more time consuming than it already is.

Another unexplored territory are different possible threshold values for the method used to avoid over-fitting. This likely doesn’t have huge effect on the performance but it is still worth mentioning.

Computing the trend, via the SMA method, required a certain window size. This window size represents the amount of previous closing prices used to compute the current SMA. This way we compute this value is highly influential on our predictions, as it is the way we acquire the residuals, which the model is trained on. This influence however would be the same for any such techniques and as this is outside the scope of our research question it wasn’t considered as a parameter to be tweaked.

The sliding window technique is also not without its flaws. Because of the bootstrap problem (outlined in 3. Methods and experiments) the further into the feature we tried to predict the error was only getting larger and larger, thus we decided to stop at $N = 5$ predictions. This number is the point at which the model has more estimates, than actual data, to do the next estimate.

These are one of the main issues we decided not tackle head on. Given the time constraints we tried to focus mainly on the MLP, and try not to spend too much time optimising those statistical tricks we used.

There was a problem, that we could not identify, with the data retrieval as we were not able to use data before 2021. This constraint of data might have been a slight detriment to the model’s performance, as more training and validation data would have resulted in a better fit.

6 Conclusion

The best set of hyperparameters to use for construction of an MLP, that is then used as an intrinsic part of a pipeline, designed to predict the next closing price of Apple Inc.’s stock is the network with 1 layer, 8 neurons, learning rate of 0.0035, and batch size of 3 (first entry in Table 1).

References

- Yahoo. (2024). Yahoo finance. <https://finance.yahoo.com>
Aroussi, R. (2024). yfinance [Python library. Version 0.2.44]. <https://github.com/ranaroussi/yfinance>

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Jia, Y., Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, ... Xiaoqiang Zheng. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems [Software available from tensorflow.org]. <https://www.tensorflow.org/>

Chollet, F., et al. (2015). Keras.

Kingma, D., & Ba, J. (2019). Adam: A method for stochastic optimization.

Chen, X. (2023). Stock price prediction using machine learning strategies. <https://doi.org/10.54691/bcpbm.v36i.3507>

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)

A Code Repository

The code repository used for this project can be found at <https://github.com/Yppilf/NeuralNetwork.project.git>.

B ChatGPT usage

ChatGPT is used for the following aspects of this report: the creation of LaTeX formulas, the creation of visualization plots, the crafting of tables and the crafting of the flowchart. It is also used for a final grammar check, via the prompt: "<article>List all grammatical errors from severe to mild".

C Statistical Analysis

	Neurons Layer 1	Neurons Layer 2	Learning Rate	Batch Size	Number of Layers
Cor. Coeff.	0.19	-0.06	0.18	-0.09	-0.16

Table 2: The correlation coefficients of the hyperparameters with respect to the MAE.

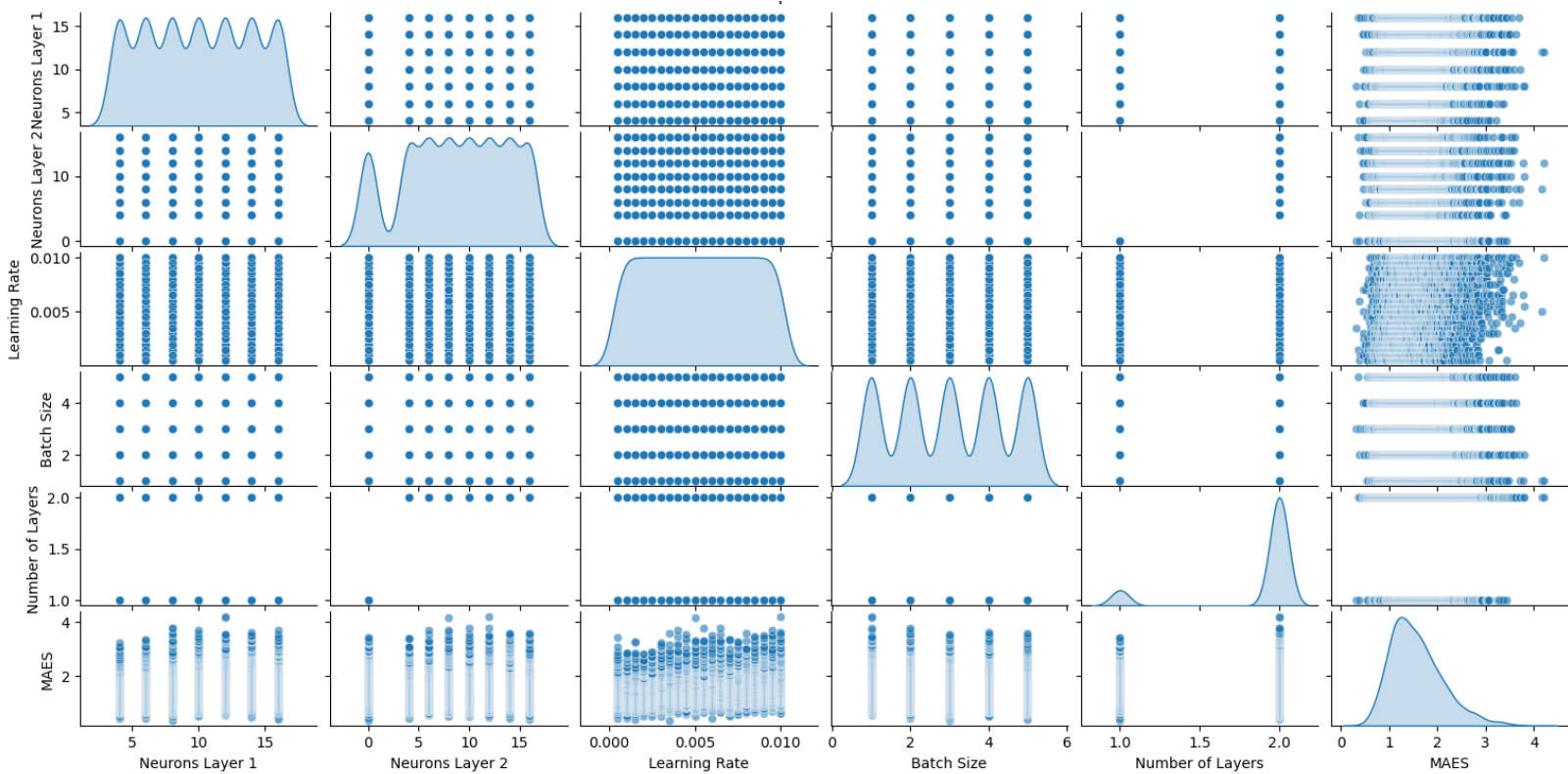


Figure 6: Matrix Scatterplot of the explored hyperparameters. This is a visual representation of the relationship between every possible parameter pair. On the diagonal one can see the distribution of the values of each parameter.

	Neurons Layer 1	Neurons Layer 2	Learning Rate	Batch Size	Number of Layers	MAES
Neurons Layer 1	None	1.0	1.0	1.0	1.0	0.0
Neurons Layer 2	1.0	None	1.0	1.0	0.0	0.000017
Learning Rate	1.0	1.0	None	1.0	1.0	0.0
Batch Size	1.0	1.0	1.0	None	1.0	0.0
Number of Layers	1.0	0.0	1.0	1.0	None	0.0
MAES	0.0	0.000017	0.0	0.0	0.0	None

Table 3: p-values of the correlation coefficients in the heat map (Figure 3).

Parameter	Min Value	Expected Value (Mean)	Max Value
Neurons Layer 1	4.0	10.0	16.0
Neurons Layer 2	0.0	8.75	16.0
Learning Rate	0.0005	0.00525	0.0100
Batch Size	1.0	3.0	5.0
Number of Layers	1.0	1.875	2.0
MAES	0.32320	1.570749	4.205370

Table 4: This table represents the min, max, and mean values of the hyperparameters used to explore the topology of the parameter space.